

Exploring User Experience of Automatic Documentation Tools

Shengnan Wu, Yangfan Zhou, Xin Wang

School of Computer Science, Fudan University

Shanghai Key Laboratory of Intelligent Information Processing

Shanghai, China

{19110240045,zyf,xinw}@fudan.edu.cn

ABSTRACT

Currently, researches about automatic documentation tools seldom pay attention to user experience. In attempt to explore how existing works consider user experience of automatic documentation tools and user experience challenges automatic documentation tools are faced with, we qualitatively analyze evaluation sections in 21 papers. We find (1) User experience of automatic documentation tools is usually considered as the supplementary for document quality. (2) Automatic documentation tools are faced with three user experience challenges. Developers do not trust the generated documents. High expectations towards automatic documentation tools also hurt user experience. How developers work and acquire information presents another challenge to the design of automatic documentation tools. Apart from identifying three user experience related challenges, this paper also benefits none SE professionals who are interested in automatic documentation tools by selecting representative papers and summarizing how evaluations are conducted.

CCS CONCEPTS

• Human-centered computing → Empirical studies in HCI.

KEYWORDS

Developers, Automatic Documentation Tools, User Experience

ACM Reference Format:

Shengnan Wu, Yangfan Zhou, Xin Wang. 2021. Exploring User Experience of Automatic Documentation Tools. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '21 Extended Abstracts)*, May 8–13, 2021, Yokohama, Japan. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3411763.3451606>

1 INTRODUCTION

The history of HCI is the process that the concept *user experience* emerges and gains the attention of researchers [3]. Currently user experience of general tools gets more research attention, but that of professional tools, for example, automatic documentation tools for developers, is usually understudied.

Some investigation suggests that developers face heavy workloads and they spend about 58% of their working time on comprehending codes [40]. Inaccuracy and outdated-ness of software process documents, which exist universally, deteriorate understandability and maintainability of codes and thus add to workloads of developers [7, 37]. Although automatic documentation tools are proposed to solve this problem, very little sign of extensive use can be witnessed. Automatic documentation tools can take their effect only when developers decide to make use of them, which is a choice closely related to user experience that developers have to make. Existing works mainly emphasize the quality of generated software documents as well as advancement of algorithms, and user experience is usually neglected [17, 42].

We attempt to examine the user experience of automatic documentation tools and answer: RQ1: How existing works consider user experience of automatic documentation tools? RQ2: What are the user experience challenges automatic documentation tools are faced with? We conduct qualitative analysis of the evaluation sections in 21 related papers from mainstream conferences in Software Engineering (SE). The rationale behind our research method is how an automatic documentation tool is evaluated more or less reflects the expectations towards it. We find evaluation focuses of automatic documentation tools still remain algorithm oriented. Only 3 in our 21 selected papers evaluate automatic documentation tools by how they help developers. User experience related factors are not frequently dealt with during evaluations. All 21 selected papers do not include specific sections evaluating user experience. User experience of automatic documentation tools is usually considered as the supplementary for document quality. The investigation towards user experience of automatic documentation tools now reaches where some works take usability into consideration but further investigation can seldom be witnessed. Automatic documentation tools are faced with three user experience challenges. Developers do not trust the generated documents. High expectations toward automatic documentation tools also affect user experience. How developers work and acquire information presents another challenge to the design of automatic documentation tools.

Unlike existing works which mainly focus on improvement of algorithms, we attempt to bring more research attention to user experience of automatic documentation tools. This paper contributes to HCI community by identifying three user experience related challenges. It selects representative papers of automatic documentation and summarizes how evaluations are conducted, which also benefits none SE professionals who are interested in automatic documentation tools. We also call for closer collaboration between SE and HCI.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '21 Extended Abstracts, May 8–13, 2021, Yokohama, Japan

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8095-9/21/05...\$15.00

<https://doi.org/10.1145/3411763.3451606>

2 BACKGROUND

This section introduces why developers need automatic documentation tools and how can automatic documentation tools help them. Comprehending code is one of the important sources of workloads for developers and it takes about 58% of working time [40]. Developers usually depend on software process documents to understand codes. During developing tasks, they visit comments more frequently instead of the source codes [7]. If no accurate documents are available, they are even willing to go through the outdated ones [18]. For the convenience of information acquiring, developers value the accuracy and completeness of documents [1]. However, current software process documents, typically generated and maintained manually, usually suffer from incompleteness and outdatedness [37]. This may due to the fact that document related tasks are less creative so developers are less motivated to complete them and the limited time that developers can contribute to documents. Automatic documentation tools are proposed to solve this problem. They relieve document related workloads and enhance the quality of software process documents.

Automatic documentation tools can help developers complete a variety of document related tasks. To list some for examples, they take source codes as input, summarize the responsibilities of Java classes and output comments for Java codes [27]. During testing, they generate natural language summaries for auto-generated test cases [30]. When developers submit codes, they give commit messages from change sets [5, 19], generate release notes from code summaries [28], and thus facilitate the management of different versions of the same software.

Automatic documentation tools usually make use of machine learning or deep neural networks. At first, they typically follow extractive approaches. For example, Ying and Robillard use a classifier to decide whether a line of code should be written into a code case and by this way complete code case summarization [41]. Later, many automatic documentation tools begin to follow generative approaches. For example, McBurney and McMillan analyze how Java methods are invoked, collect keywords from the context of a method, and adopt a generation system to assemble those keywords into sentences to complete automatic code summarization for Java methods [26]. Recently, a line of researches consider source codes and documents as different languages and complete automatic documentation by translation models. For example, Jiang et al. adopt Neural Machine Translation to automatically generate commit messages from diffs [14].

3 SELECTING STUDY OBJECTS

This section illustrates why we choose to conduct a qualitative analysis of evaluation sections and how the study objects are selected.

In order to answer our RQs, we choose to qualitatively analyze the evaluation sections of papers about automatic documentation. The rationale behind our research method is: How evaluations are conducted somehow unveils the expectations researchers hold for their automatic documentation tools and the evaluation results also give clues to user experience issues.

As we explore the user experience of automatic documentation tools by analyzing evaluation sections, we need a pool of papers as our study objects. For the reliability of findings, papers in that pool

should be of high quality and cover as many important automatic documentation tools, although exhaustiveness is not necessary.

We take a snowball-sampling-alike approach. First we select all full papers proposing automatic documentation tools published in ICSE, FSE and ASE during the latest five events¹ as our seeds. Seeds are papers selected for our analysis and can also help us find other similar papers. To the best of our knowledge, we get 11 seeds. We achieve this by reading through all papers in the three conferences during the given 5 years from ACM and IEEE libraries. Then we build our pool of papers by two searching and filtering processes.

In the first process, to reach a more comprehensive coverage of automatic documentation tools, we expand our pool by citation relations and recommendations given by Google Scholar. We select more papers from papers cite or are cited by our seeds as well as related papers of our seeds recommended by Google Scholar. Papers selected must meet three criteria: They propose and evaluate automatic documentation tools. They are full papers published in mainstream conferences of SE, because those papers must be qualified to represent state of the art. For the same reason they must have been cited by other papers.

In the following process, we turn to human evaluation to ensure the quality of selected papers. A graduate student who has been in the field of SE over 4 years examines all the selected papers and eliminates papers that are too old or inadequate to represent state of the art.

After the two processes, we find other 10 papers besides our seeds and thus get a pool of 21 papers. Specifically, we do not select any short papers for the evaluation sections in short papers are usually comparatively brief. We are concerned they may not provide enough information and thus lead to biased conclusions. The basic information of those 21 papers is shown in Table 1.

All selected papers are very recent. Specifically, more than half of them (62%) are published during the last 5 years. All selected papers are of high quality. They all are published in mainstream conferences and journals. Specifically, 13 out of 21 papers (62%) are published in ICSE, ASE, and FSE. All selected papers show importance to advancement of automatic documentation. They have been cited at least 6 times and 6 of them have been cited more than 100 times, which implies the trust and approval the whole community gives to those papers. The importance of those papers is unveiled more considering automatic documentation is not among the most popular research topics in SE, and those papers get cited 84 times on average. The 21 papers we select are very recent, important as well as of high quality. Automatic documentation tools proposed in those papers can represent state of the art. Those papers can be used for further analysis.

4 DATA ANALYSIS AND PRELIMINARY FINDINGS

This section describes how we conduct our analysis and the preliminary results towards user experience of automatic documentation tools. First we address RQ1. We explore how user experience of automatic documentation tools is considered by analyzing how evaluations of automatic documentation tools are conducted. Next we address RQ2. We identify user experience challenges automatic

¹ICSE (2016-2020), FSE (2016-2020), ASE (2015-2019)

Table 1: Basic Information of Selected Papers.

paper	Authors	Published Time	Source	Times Cited (till 2021/01/05)
[42]	Zhang et al.	2020	ICSE	7
[17]	LeClair et al.	2019	ICSE	30
[8]	Hao et al.	2019	ICSE	11
[21]	Liu et al.	2019	FSE	6
[22]	Liu et al.	2019	ASE	10
[39]	Wan et al.	2018	ASE	62
[33]	Rodeghero et al.	2017	ICSE	28
[14]	Jiang et al.	2017	ASE	95
[36]	Uddin and Khomh	2017	ASE	32
[30]	Panichella et al.	2016	ICSE	78
[29]	Moreno et al.	2016	TSE	44
[6]	Di Sorbo et al.	2016	FSE	171
[20]	Linares-Vásquez et al.	2016	ISSTA	23
[23]	Lotufo et al.	2015	ESE	97
[25]	McBurney and McMillan	2014	ICPC	153
[28]	Moreno et al.	2014	FSE	71
[5]	Cortés-Coy et al.	2014	SCAM	104
[32]	Rastkar et al.	2014	TSE	125
[43]	Zhong and Su	2013	OOPSLA	60
[27]	Moreno et al.	2013	ICPC	216
[35]	Sridhara et al.	2010	ASE	351

documentation tools are faced with by examining qualitative feedbacks in the evaluation sections. Detailed findings are presented in the following 2 subsections.

4.1 How User Experience of Automatic Documentation Tools is Considered

How an automatic documentation tool is evaluated reflects the expectations towards it. When an automatic documentation tool is proposed and presented in a paper, its qualities that are valued most get more attention in the evaluation, and qualities that are considered less important usually get neglected. In order to understand the attitude towards user experience of automatic documentation tools and answer RQ1, we take 21 papers selected in section 3 as our study objects. We extract how evaluations are done in Table 2 and Table 3. To improve readability, Table 2 shows general-level information, and Table 3 shows specifics.

Experiments are widely used in evaluation sections of our selected papers. 20 out of all 21 (95%) papers use experiments to evaluate their proposed automatic documentation tools and among them, 60% use experiments as only evaluation method. Experiments are usually conducted to collect data that is objective and quantitative. For example, performance of an algorithm is typically measured by experiments. However, user experience is a complex fabric of feelings, thoughts, and actions [9], which is highly subjective and dynamic [16]. User experience is closely related to the fulfillment of psychological needs and backgrounds of the users [10], which can not be measured by quantitative methods alone. So, qualitative methods play an important role in measuring user experience. According to the findings by Vermeeren et al., about a third of works

evaluate user experience by both qualitative and quantitative methods and another third by qualitative methods only [38]. The fact that conducting experiments becomes the dominant evaluation method in the selected 21 papers may imply that user experience is not considered as one of the top priorities of automatic documentation tools.

That user experience may not be considered as one of the top priorities of automatic documentation tools is also confirmed by that all the selected 21 papers do not include specific sections dedicated to user experience evaluation. As user experience can not be evaluated by the same way of measuring algorithmic performance of automatic documentation tools, another user experience evaluation section using different methods is usually needed. No specific user experience evaluation sections included may further indicate that user experience receives little attention in the evaluation of automatic documentation tools proposed in the 21 papers.

According to how evaluations are conducted in the selected 21 papers, user experience of automatic documentation tools may not be the evaluation emphasis. To get more detailed information and understand how user experience related factors are dealt with in those 21 papers, we turn to evaluation perspectives and evaluation focuses, which are summarized from evaluation sections and shown in Table 3.

In total, only 3 papers evaluate automatic documentation tools by helpfulness to developers. By contrast, 12 out of 21 (57%) papers evaluate automatic documentation tools proposed by them via performance of algorithms and quality of the generated documents. The majority of selected papers (57%) value effectiveness of the methods used and pay more attention to advancement of algorithms. Little attention is given to users. In fact, 3 of the selected

Table 2: Basic Information of Evaluations in 21 Selected Papers. UX sections refer to specific sections dedicated to user experience evaluation.

Paper	Aim of the Proposed Tool	Evaluation Method (s)	UX Sections
[42]	generating code summaries	experiments	not included
[17]	generating code summaries	experiments	not included
[8]	aggregating test reports	experiments, questionnaires, interviews	not included
[21]	generating summaries for API classes	experiments, interviews	not included
[22]	generating pull request descriptions	experiments	not included
[39]	generating code summaries	experiments	not included
[33]	extracting materials needed for user stories	experiments	not included
[14]	generating commit messages	experiments, questionnaires	not included
[36]	summarizing API reviews	experiments	not included
[30]	generating summaries for test cases	experiments	not included
[29]	generating release notes	experiments, questionnaires, field studies	not included
[6]	generating app improvement reports	experiments, questionnaires	not included
[20]	generating database related comments	experiments, interviews	not included
[23]	summarizing bug reports	experiments	not included
[25]	generating summaries for Java methods	experiments	not included
[28]	generating release notes	experiments, questionnaires	not included
[5]	generating commit messages	questionnaires	not included
[32]	summarizing bug reports	experiments, interviews	not included
[43]	detecting errors in API documents	experiments	not included
[27]	generating summaries for Java classes	experiments	not included
[35]	generating comments for Java methods	experiments	not included

Table 3: Specifics of Evaluations in 21 Selected Papers.

Paper	Evaluation Perspective	Evaluation Focuses
[42]	performance of algorithms	generated summaries' similarity to the references
[17]	performance of algorithms	generated summaries' similarity to the references
[8]	potential of proposed tool	aggregating ability, developers' efficiency improvement
[21]	quality of generated documents	completeness, conciseness, understand-ability, usefulness
[22]	performance of algorithms	textual and perceived effectiveness
[39]	performance of algorithms	effectiveness of proposed algorithm and its components
[33]	performance of algorithms	ability of extracting information
[14]	performance of algorithms	generated messages' similarity to the references
[36]	helpfulness to developers	documents helping with tasks, usefulness of the tool
[30]	helpfulness to developers	developers' efficiency improvement
[29]	potential of proposed tool	its strengths, quality of generated release notes
[6]	helpfulness to developers	improvement in user-understanding
[20]	potential of proposed tool	quality of generated comments, willingness to use it
[23]	potential of proposed tool	quality and usefulness of generated summaries
[25]	potential of proposed tool	quality and usefulness of generated summaries
[28]	quality of generated documents	completeness, accuracy, importance
[5]	quality of generated documents	conciseness, completeness, expressiveness
[32]	potential of proposed tool	quality, usefulness, user-satisfaction of generated summaries
[43]	performance of algorithms	efficiency and accuracy in detecting errors
[27]	quality of generated documents	informativeness, conciseness, expressiveness
[35]	quality of generated documents	accuracy, informativeness, conciseness

papers claim that their contribution mainly lies in proposing a new technique, which is technology oriented instead of user oriented. According to those 21 papers, user experience related factors are not dealt with frequently.

Less than half of papers (43%, 9 out of 21) take user experience related factors into consideration when conducting evaluations. Apart from advancement of algorithms, they also pay attention to how the proposed tools help developers complete tasks and whether developers are willing to adopt the proposed automatic documentation tools in work settings. Specifically, they emphasize efficiency improvements of developers, usefulness, willingness to use the proposed tool and satisfaction. Those are typical measurements of usability [13]. Although has been used in actual design for years, the concept of user experience is still rather vague [11]. It intertwines with the concept of usability and the latter usually is included in the former [38]. After years of development, measuring of emotions has also become an important part of user experience [34]. Based on the 21 papers, usability has been considered in some works but other parts of user experience, like emotions, are still understudied when evaluating automatic documentation tools.

In summary, according to the 21 selected papers, user experience receives very limited research attention. User experience related factors are not dealt with frequently in the evaluation of automatic documentation tools. When conducting evaluations, the advancement of algorithms is valued and user experience is considered not among top priorities of automatic documentation tools. Some factors related to usability have been noticed and explored besides advancement of algorithms in 9 out of 21 selected papers. But none of the selected papers pay any attention to the emotion related part of user experience. Our selected 21 papers support that user experience of automatic documentation tools has usually been considered as a supplementary for document quality. The investigation towards user experience of automatic documentation tools now reaches where some works take usability into consideration but further investigation can seldom be witnessed.

4.2 User Experience Challenges Automatic Documentation Tools Faced with

Although no specific sections dedicated to user experience are included in all our selected papers, 9 of them contain qualitative user feedbacks, which scatter in the evaluation sections. We identify user experience challenges that automatic documentation tools proposed in the selected 21 papers are faced with based on those qualitative feedbacks.

One of the frequently reported issues in those 9 papers is that developers do not trust the documents generated by automatic documentation tools. Automatic documentation tools usually adopt deep neural networks, which are generally considered as black boxes. Developers can hardly trust the documents without knowing how they are generated. A participant expresses his distrust in a very direct way:

"I would never trust an automated summary, and would always need to refer to the original" [23]. Another participant evaluating a test report summarizing tool also states that screenshots *"are open to various interpretations"* and *"can't tell where's the problem"* [8]. The distrust towards the generated documents deteriorates the user

experience of the corresponding automatic documentation tools and makes the tools less useful.

Beside the issue of distrust, people usually have high expectations towards highly automated or smart tools. No matter how smart the used algorithms are, automatic documentation tools can not guarantee 100% accurate and effective. They might generate documents that are difficult to understand or different from the somewhat ideal ones developers expect. Over half (5 out of 9, 56%) of papers containing qualitative user feedbacks admit that some participants have trouble understanding the generated documents. 17% of automatic commit messages are hard to read and understand [5]. As a participant feels the summarized reports confusing, he states:

"...I would improve the report design with a more readable interface and by including extra information..." [6].

Another participant also expresses a similar idea when he finds some items in the generated release notes do not have a textual description motivating the change.

"Every item needs a description to be useful. For example, the Added Components section needs a description of each component, and the Modified Components section needs a description of what is the meaning of the modification" [29].

The gap between the expectations and the perceived quality of generated documents may lead to frustration and eventually harms the user experience of automatic documentation tools.

Developers seldom read software documents line by line. Maybe because of heavy workloads, they need to acquire information from documents in a speedy way. However, some automatic documentation tools fail to take that into consideration. How developers work and use software documents presents another challenge of user experience for automatic documentation tools. A developer participating in the evaluation of a tool summarizing API reviews notes how the generated documents contradict with his information-acquiring habits:

"I think the categories need to be in alphabetical order. It makes it hard when I was comparing one API to another because I had to ctrl-f to find category" [36].

This is not the only case. Participants also suggest that a test report aggregation tool does not give *"well-structured description"* and it would improve by *"highlighting important parts of screenshots"* [8].

In summary, according to 9 papers containing qualitative feedbacks, automatic documentation tools are faced with three user experience related challenges. Developers do not trust the generated documents. High expectations toward automatic documentation tools also affect user experience. How developers work and acquire information presents another challenge to the design of automatic documentation tools.

5 DESIGN IMPLEMENTATION

As presented in section 4, automatic documentation tools are faced with three user experience related challenges.

The challenge of distrust is not unique for automatic documentation tools. It is common for many smart systems and tools. For users, those tools are like black boxes and only explanations of how those tools work can provide some transparency [12]. Automatic documentation tools can provide explanations about how

the documents are generated to users. There are many strategies of providing explanations. Explanations that can help users decide whether the output is correct and trustworthy prove to be more effective [31].

Management of user expectation is also necessary for automatic documentation tools which are smart and highly automatic. Expectation management originates from the field of marketing. Bhattacherjee and Anol find that expectations of users also affect their user experience towards information systems and tools [4]. Users often have overly high expectations towards smart or intelligent tools [24], which can eventually hurt the user experience. Currently, expectation management is usually done by information filtering before the actual usage behavior [15]. For smart systems and tools, expectation management can also be done by manipulating key parameters, which changes how tools behave [15].

Understanding users should be the first step of designing. Automatic documentation tools can benefit from a comprehensive study of developers. For example, developers usually acquire information by skip reading, and they pay more attention to function names [2]. A searching functionality may help developers find information from the generated documents and enhance user experience of automatic documentation tools.

6 CONCLUSION

This paper examines the user experience of state of the art automatic documentation tools by qualitatively analyzing 21 representative papers. According to the 21 papers selected as study objects, advancements of algorithms are valued in the evaluation of automatic documentation tools. User experience has usually been considered as a supplementary to document quality. The investigation towards user experience of automatic documentation tools now reaches where some works take usability into consideration but further investigation can seldom be witnessed. Automatic documentation tools are faced with three user experience challenges. Developers do not trust the generated documents. High expectations towards automatic documentation tools also affect user experience. How developers work and acquire information presents another challenge to the design of automatic documentation tools. Our paper contributes to HCI community by identifying three user experience related challenges. It selects representative papers of automatic documentation and summarizes how evaluations are conducted, which could benefit none SE professionals who are interested in automatic documentation tools. We also emphasize more research attention on the user side instead of the technology side, and call for closer collaboration between SE and HCI.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (Project No. 61971145).

REFERENCES

- [1] E. Aghajani, C. Nagy, O. L. Vega-Márquez, M. Linares-Vásquez, L. Moreno, G. Bavota, and M. Lanza. 2019. Software Documentation Issues Unveiled. In *2019 IEEE/ACM 41st International Conference on Software Engineering*. IEEE / ACM, Montreal, QC, Canada, 1199–1210.
- [2] Ameer Armaly, Paige Rodeghero, and Collin McMillan. 2017. A comparison of program comprehension strategies by blind and sighted programmers. *IEEE Transactions on Software Engineering* 44, 8 (2017), 712–724.
- [3] Liam J. Bannon. 1995. From Human Factors to Human Actors: The Role of Psychology and Human-Computer Interaction Studies in System Design. In *Readings in Human-Computer Interaction*, RONALD M. BAECKER, JONATHAN GRUDIN, WILLIAM A.S. BUXTON, and SAUL GREENBERG (Eds.). Morgan Kaufmann, San Francisco, California, USA, 205–214.
- [4] Anol Bhattacherjee. 2001. Understanding information systems continuance: an expectation-confirmation model. *MIS quarterly* 25, 3 (2001), 351–370.
- [5] Luis Fernando Cortés-Coy, Mario Linares-Vásquez, Jairo Aponte, and Denys Poshyvanyk. 2014. On automatically generating commit messages via summarization of source code changes. In *2014 IEEE 14th International Working Conference on Source Code Analysis and Manipulation*. IEEE Computer Society, Victoria, BC, Canada, 275–284.
- [6] Andrea Di Sorbo, Sebastiano Panichella, Carol V Alexandru, Junji Shimagaki, Corrado A Visaggio, Gerardo Canfora, and Harald C Gall. 2016. What would users change in my app? summarizing app reviews for recommending software changes. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, Seattle, WA, USA, 499–510.
- [7] Golará Garousi, Vahid Garousi-Yusifoglu, Guenther Ruhe, Junji Zhi, Mahmoud Mousavi, and Brian Smith. 2015. Usage and usefulness of technical software documentation: An industrial case study. *Information and Software Technology* 57 (2015), 664–682.
- [8] Rui Hao, Yang Feng, James A Jones, Yuying Li, and Zhenyu Chen. 2019. Ctrast: Crowdsourced test report aggregation and summarization. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE / ACM, Montreal, QC, Canada, 900–911.
- [9] Marc Hassenzahl. 2013. *User experience and experience design*. The Interaction Design Foundation. Retrieved Feb 24, 2021 from <https://www.interaction-design.org/literature/book/the-encyclopedia-of-human-computer-interaction-2nd-ed/user-experience-and-experience-design>
- [10] Marc Hassenzahl, Sarah Diefenbach, and Anja Göritz. 2010. Needs, affect, and interactive products—Facets of user experience. *Interacting with computers* 22, 5 (2010), 353–362.
- [11] Marc Hassenzahl and Noam Tractinsky. 2006. User experience—a research agenda. *Behaviour & information technology* 25, 2 (2006), 91–97.
- [12] Jonathan L Herlocker, Joseph A Konstan, and John Riedl. 2000. Explaining collaborative filtering recommendations. In *Proceedings of the 2000 ACM conference on Computer supported cooperative work*. ACM, Philadelphia, PA, USA, 241–250.
- [13] Kasper Hornbæk and Effie Lai-Chong Law. 2007. Meta-analysis of correlations among usability measures. In *Proceedings of the SIGCHI conference on Human factors in computing systems*. ACM, San Jose, California, USA, 617–626.
- [14] Siyuan Jiang, Ameer Armaly, and Collin McMillan. 2017. Automatically generating commit messages from diffs using neural machine translation. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Computer Society, Urbana, IL, USA, 135–146.
- [15] Rafal Kocielnik, Saleema Amershi, and Paul N Bennett. 2019. Will you accept an imperfect ai? exploring designs for adjusting end-user expectations of ai systems. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, Glasgow, Scotland, UK, 1–14.
- [16] Effie Lai-Chong Law, Virpi Roto, Marc Hassenzahl, Arnold POS Vermeeren, and Joke Kort. 2009. Understanding, scoping and defining user experience: a survey approach. In *Proceedings of the SIGCHI conference on human factors in computing systems*. ACM, Boston, MA, USA, 719–728.
- [17] Alexander LeClair, Siyuan Jiang, and Collin McMillan. 2019. A neural model for generating natural language summaries of program subroutines. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE / ACM, Montreal, QC, Canada, 795–806.
- [18] Timothy C Lethbridge, Janice Singer, and Andrew Forward. 2003. How software engineers use documentation: The state of the practice. *IEEE software* 20, 6 (2003), 35–39.
- [19] Mario Linares-Vásquez, Luis Fernando Cortés-Coy, Jairo Aponte, and Denys Poshyvanyk. 2015. Changelog: A tool for automatically generating commit messages. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 2. IEEE Computer Society, Florence, Italy, 709–712.
- [20] Mario Linares-Vásquez, Boyang Li, Christopher Vendome, and Denys Poshyvanyk. 2016. Documenting database usages and schema constraints in database-centric applications. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*. ACM, Saarbrücken, Germany, 270–281.
- [21] Mingwei Liu, Xin Peng, Andrian Marcus, Zhenchang Xing, Wenkai Xie, Shuangshuang Xing, and Yang Liu. 2019. Generating query-specific class API summaries. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Tallinn, Estonia, 120–130.
- [22] Zhongxin Liu, Xin Xia, Christoph Treude, David Lo, and Shanping Li. 2019. Automatic generation of pull request descriptions. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, San Diego, CA, USA, 176–188.
- [23] Rafael Lotufo, Zeeshan Malik, and Krzysztof Czarnecki. 2015. Modelling the ‘hurried’ bug report reading process to summarize bug reports. *Empirical Software Engineering* 20, 2 (2015), 516–548.

- [24] Ewa Luger and Abigail Sellen. 2016. "Like Having a Really Bad PA" The Gulf between User Expectation and Experience of Conversational Agents. In *Proceedings of the 2016 CHI conference on human factors in computing systems*. ACM, San Jose, CA, USA, 5286–5297.
- [25] Paul W McBurney and Collin McMillan. 2014. Automatic documentation generation via source code summarization of method context. In *Proceedings of the 22nd International Conference on Program Comprehension*. ACM, Hyderabad, India, 279–290.
- [26] Paul W McBurney and Collin McMillan. 2015. Automatic source code summarization of context for java methods. *IEEE Transactions on Software Engineering* 42, 2 (2015), 103–119.
- [27] Laura Moreno, Jairo Aponte, Giriprasad Sridhara, Andrian Marcus, Lori Pollock, and K Vijay-Shanker. 2013. Automatic generation of natural language summaries for java classes. In *2013 21st International Conference on Program Comprehension (ICPC)*. IEEE Computer Society, San Francisco, CA, 23–32.
- [28] Laura Moreno, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrian Marcus, and Gerardo Canfora. 2014. Automatic generation of release notes. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, Hong Kong, China, 484–495.
- [29] Laura Moreno, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrian Marcus, and Gerardo Canfora. 2016. ARENA: an approach for the automated generation of release notes. *IEEE Transactions on Software Engineering* 43, 2 (2016), 106–127.
- [30] Sebastiano Panichella, Annibale Panichella, Moritz Beller, Andy Zaidman, and Harald C Gall. 2016. The impact of test case summaries on bug fixing performance: An empirical investigation. In *Proceedings of the 38th International Conference on Software Engineering*. ACM, Austin, TX, USA, 547–558.
- [31] Emilee Rader, Kelley Cotter, and Janghee Cho. 2018. Explanations as mechanisms for supporting algorithmic transparency. In *Proceedings of the 2018 CHI conference on human factors in computing systems*. ACM, Montreal, QC, Canada, 1–13.
- [32] Sarah Rastkar, Gail C Murphy, and Gabriel Murray. 2014. Automatic summarization of bug reports. *IEEE Transactions on Software Engineering* 40, 4 (2014), 366–380.
- [33] Paige Rodeghero, Siyuan Jiang, Ameer Armaly, and Collin McMillan. 2017. Detecting user story information in developer-client conversations to generate extractive summaries. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE / ACM, Buenos Aires, Argentina, 49–59.
- [34] Andreas Sonderegger, Klaus Heyden, Alain Chavallaz, and Juerger Sauer. 2016. AniSAM & AniAvatar: Animated visualizations of affective states. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. ACM, San Jose, CA, USA, 4828–4837.
- [35] Giriprasad Sridhara, Emily Hill, Divya Muppaneni, Lori Pollock, and K Vijay-Shanker. 2010. Towards automatically generating summary comments for java methods. In *Proceedings of the IEEE/ACM international conference on Automated software engineering*. ACM, Antwerp, Belgium, 43–52.
- [36] Gias Uddin and Foutse Khomh. 2017. Automatic summarization of API reviews. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Computer Society, Urbana, IL, USA, 159–170.
- [37] Gias Uddin and Martin P Robillard. 2015. How API documentation fails. *IEEE Software* 32, 4 (2015), 68–75.
- [38] Arnold POS Vermeeren, Effie Lai-Chong Law, Virpi Roto, Marianna Obrist, Jettie Hoonhout, and Kaisa Väänänen-Vainio-Mattila. 2010. User experience evaluation methods: current state and development needs. In *Proceedings of the 6th Nordic conference on human-computer interaction: Extending boundaries*. ACM, Reykjavik, Iceland, 521–530.
- [39] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. ACM, Montpellier, France, 397–407.
- [40] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E Hassan, and Shaping Li. 2017. Measuring program comprehension: A large-scale field study with professionals. *IEEE Transactions on Software Engineering* 44, 10 (2017), 951–976.
- [41] Annie TT Ying and Martin P Robillard. 2013. Code fragment summarization. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. ACM, Saint Petersburg, Russian Federation, 655–658.
- [42] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, and Xudong Liu. 2020. Retrieval-based neural source code summarization. In *ICSE '20: 42nd International Conference on Software Engineering*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, Seoul, South Korea, 1385–1397.
- [43] Hao Zhong and Zhendong Su. 2013. Detecting API documentation errors. In *Proceedings of the 2013 ACM SIGPLAN international conference on Object oriented programming systems languages & applications*. ACM, Indianapolis, IN, USA, 803–816.